pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Resampling and the As-Of Value

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: one number per period, with its real date

The Desk Lead's Note

The monthly pack shows one 10-year yield per month, and the weekly sheet one per week. Tell me which observation stands for each, and write beside it the date it really comes from. No label on a day the market never had. Show me a week with no Friday. Some reports run on a holiday or a Saturday, so give me the value as of any date, and the date it came from. Then the 2026 month-end 10-year and 2s10s, tied to Excel. Two tested functions in the module, and keep any half-finished idea out of the commit.

No key is needed today: every number comes from the Treasury file.

Today's Work Folder

```
work = start_day(4)
Work folder pfi_course3_04_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 03 left it, 3 commits
```

Imports and the Treasury File

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
ten = treasury['10 Yr']
print(len(ten), 'rows, from', ten.index[0].date(), 'to', ten.index[-1].date())
2940 rows, from 2015-01-02 to 2026-10-02
```

The Tests Pass Before We Start

```
............   [100%]
13 passed in 0.50s
```

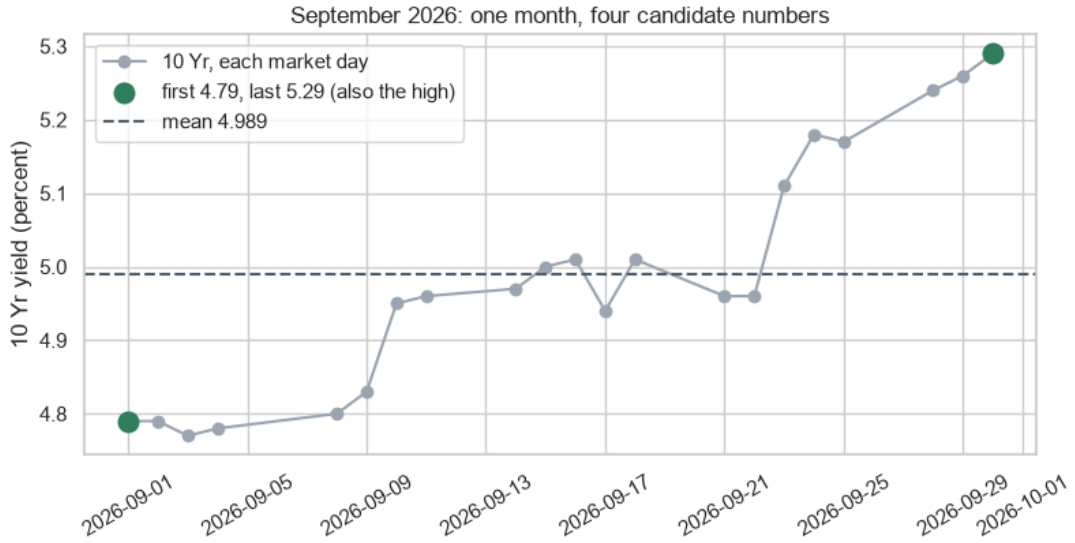
A month's number is its last observation, and its label is the date of that observation.

4.1 Which observation stands for a month

Notebook 03 settled which days the market had. Now one number has to stand for a whole month.

September, Four Ways

```
september = ten.loc['2026-09']
print(len(september), 'rows | first', september.iloc[0], '| last', september.iloc[-1],
      '| mean', round(september.mean(), 3), '| high', september.max())
21 rows | first 4.79 | last 5.29 | mean 4.989 | high 5.29
```



The mean sits on no day at all, and the first row is the month's open. The course takes the last observation: by calendar month, 'ME', and by week ending Friday, 'W-FRI'.

A month's number is its close: 5.29 on 2026-09-30, not the 4.989 mean or the 4.79 open.

Excel side by side: the open, the mean, the high

Excel side by side. The Treasury file opened in Excel: dates in A2:A2941, the 2 Yr in column I, the 5 Yr in K, the 10 Yr in M.

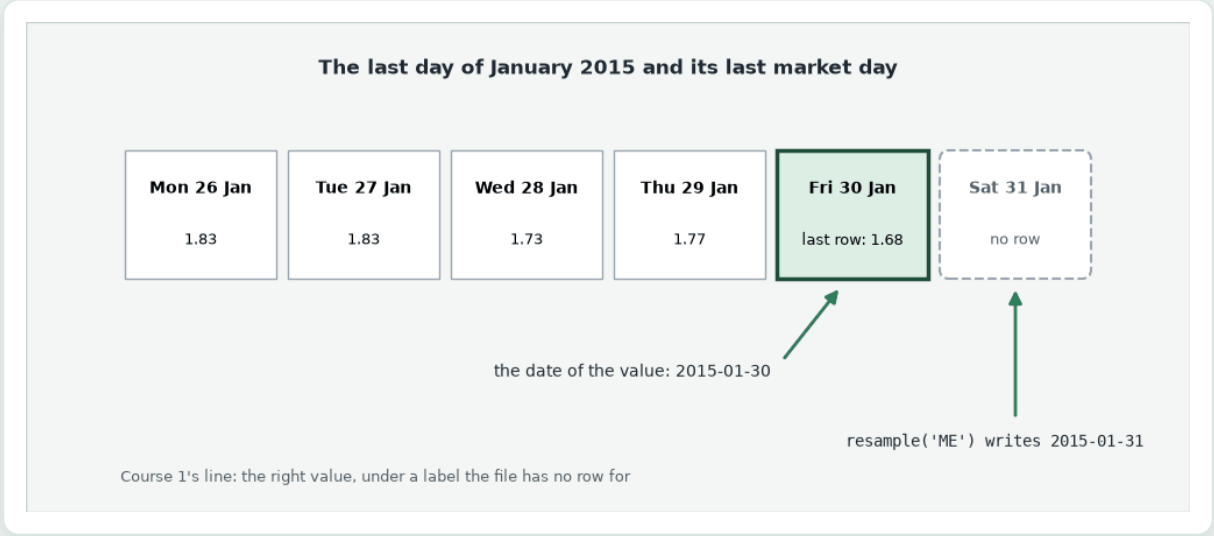
Job	Python	Result
The first row on or after 1 September	`september.iloc[0]`	4.79
The mean	`september.mean()`	4.989
The high	`september.max()`	5.29

```
The first row on or after 1 September
=XLOOKUP(DATE(2026,9,1),$A$2:$A$2941,M2:M2941,,1)
The mean
=AVERAGEIFS(M2:M2941,$A$2:$A$2941,">="&DATE(2026,9,1),$A$2:$A$2941,"<="&DATE(2026,9,30))
The high
=MAXIFS(M2:M2941,$A$2:$A$2941,">="&DATE(2026,9,1),$A$2:$A$2941,"<="&DATE(2026,9,30))
```

XLOOKUP's last argument, 1, is its match mode: the date itself or the next one after it. "AVERAGEIFS" and "MAXIFS" average, or take the largest of, the rows that meet every condition.

4.2 The label resample prints

Course 1's Treasury Yields case study took month ends with resample('ME').last(): 142 rows, each labelled by its calendar month end. It found one label, 2015-01-31, that the file has no row for.



The label is the period's calendar end, not the date of the value.

A Join on Those Labels

Q. The pack joins the 2 Yr to these labels to build a month-end 2s10s. What does it find?

```
monthly = pd.DataFrame({'10 Yr': by_resample}).join(treasury['2 Yr'])
print(monthly['2 Yr'].count(), 'of', len(monthly), 'months found a 2 Yr')
monthly.loc['2026']
99 of 142 months found a 2 Yr
```

Nothing raised an error. A month-end 2s10s built on these labels quietly loses 43 months.

The values are right and the labels are not: a join on 43 of 142 month labels finds no row.

4.3 last_per_period: each month with its own date

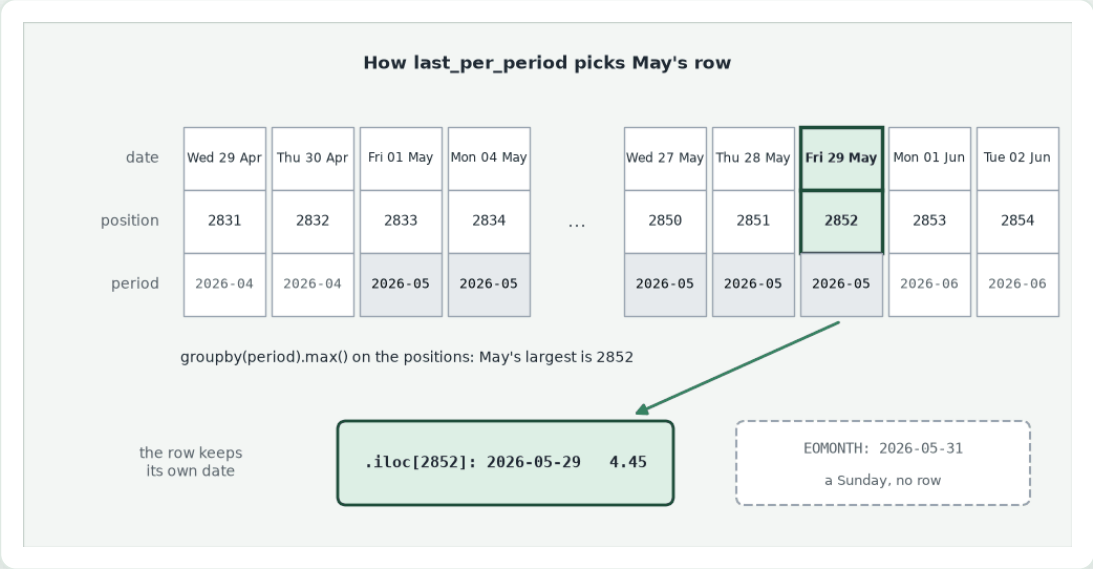
```
%%writefile -a marketdata.py
def _check_series(series: pd.Series, what: str) -> None:
    """Stop unless `series` has unique ascending dates and no missing values (a helper, not part of the API)."""
    _check_index(series.index, what)
    if series.isna().any():
        raise ValueError(f"{what} has {series.isna().sum()} missing values; drop them first with .dropna()")
def last_per_period(data: pd.Series | pd.DataFrame, freq: str) -> pd.Series | pd.DataFrame:
    """The last observation in each week or month, labelled by the date of that observation.
    Units: unchanged. Convention: freq "W-FRI" is the week ending Friday, "ME" the calendar month. Each
    period's row is the last row the data has in it, kept with its own date: September 2026 is labelled
    2026-09-30, and the week of Good Friday 2025 (no row on Friday 2025-04-18) is labelled Thursday 2025-04-17.
    pandas' resample(freq).last() labels by the period end instead. A Series must have no missing values;
    a DataFrame's rows are taken as they are, so a tenor with no value that day stays NaN.
    Raises ValueError for any other freq, or for an index that is not unique ascending dates.
    """
    periods = {"W-FRI": "W-FRI", "ME": "M"}
    if freq not in periods:
        raise ValueError(f"freq must be 'W-FRI' or 'ME', not {freq!r}")
    if isinstance(data, pd.Series):
        _check_series(data, "data")
    else:
        _check_index(data.index, "data")
    period_of_row = data.index.to_period(periods[freq])
    positions = pd.Series(range(len(data)), index=data.index).groupby(period_of_row).max()
    return data.iloc[positions.to_numpy()]
```

The course's rule is each period's last observation, labelled by the date of that observation.

Append It, Docstring First

Read both docstrings first. "_check_series" is notebook 03's "_check_index" with one more stop: no missing values. "pd.Series | pd.DataFrame" reads as notebook 02's "str | None" did: either kind goes in, and the same kind comes back.

Month ends with real dates



.to_period('M') turns each date into its month: pandas names a month 'M' as a period and 'ME' as a month end. Each row gets its position, "groupby(...).max()" keeps each month's largest, and ".iloc[...]" takes those rows with their own dates.

Month Ends With Real Dates

```
importlib.reload(md)
month_end = md.last_per_period(ten, 'ME')
month_end.loc['2026']
```

The Same Values, Every Label a Market Day

```
print(len(month_end), 'and', len(by_resample), 'rows; values equal:',
      (month_end.to_numpy() == by_resample.to_numpy()).all())
print((month_end.index != by_resample.index).sum(), 'labels differ; every label in the file:',
      month_end.index.isin(ten.index).all())
142 and 142 rows; values equal: True
43 labels differ; every label in the file: True
```

A Period It Was Not Built For

```
md.last_per_period(ten, 'QE')
ValueError: freq must be 'W-FRI' or 'ME', not 'QE'
```

"last_per_period" keeps resample's 142 values and writes the real date: every label a market day, equal to Excel's "MAXIFS" then "XLOOKUP".

Excel side by side: May 2026, whose month ends on a Sunday

Excel side by side. May 2026, whose calendar end is a Sunday, with Course 2's "EOMONTH":

Job	Python	Result
The calendar end	`by_resample.loc['2026-05'].index[0]`	2026-05-31
Its weekday, counting Monday as 1	`4.2's loop: Sunday`	7, a Sunday
The last market date on or before it	`month_end.loc['2026-05'].index[0]`	2026-05-29
Its 10 Yr	`month_end.loc['2026-05-29']`	4.45

```
The calendar end
=EOMONTH(DATE(2026,5,15),0)
Its weekday, counting Monday as 1
=WEEKDAY(EOMONTH(DATE(2026,5,15),0),2)
The last market date on or before it
=MAXIFS($A$2:$A$2941,$A$2:$A$2941,"<="&EOMONTH(DATE(2026,5,15),0))
Its 10 Yr
=XLOOKUP(MAXIFS($A$2:$A$2941,$A$2:$A$2941,"<="&EOMONTH(DATE(2026,5,15),0)),$A$2:$A$2941,$M$2:$M$2941)
```

"MAXIFS" over the dates returns the latest date that meets the condition. Filled down over one date per month, all 142 dates and 10 Yr values equal last_per_period's.

4.4 The week of Good Friday 2025

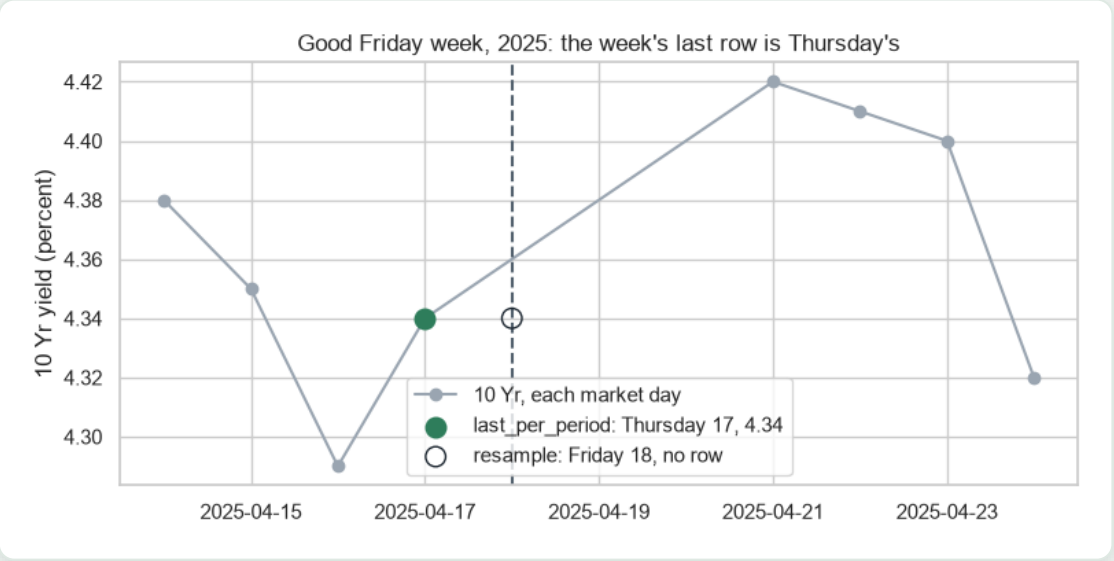
Weeks end on Friday: Course 1's Trade Blotter case study used resample('W-FRI') in an exercise. Good Friday, 2025-04-18, is one of notebook 03's missing weekdays.

That Week's Rows

```
ten.loc['2025-04-14':'2025-04-18']
date
2025-04-14   4.38
2025-04-15   4.35
2025-04-16   4.29
2025-04-17   4.34
Name: 10 Yr, dtype: float64
```

Both Labels

```
weekly = md.last_per_period(ten, 'W-FRI')
weekly_resample = ten.resample('W-FRI').last()
print(len(weekly), 'weeks')
around = weekly.loc['2025-04-11':'2025-04-25']
pd.DataFrame({'resample': weekly_resample.loc['2025-04-11':'2025-04-25'].index,
              'last_per_period': around.index, '10 Yr': around.to_numpy()})
614 weeks
   resample last_per_period  10 Yr
0 2025-04-11   2025-04-11   4.48
1 2025-04-18   2025-04-17   4.34
2 2025-04-25   2025-04-25   4.29
```



Excel side by side, and two tests for the labels

Excel side by side.

Job	Python	Result
The Friday ending a date's week	`'W-FRI'`	2025-04-16 gives 2025-04-18;
		Saturday 2025-04-19 gives 2025-04-25
That week's last market date	`around.index[1]`	2025-04-17

```
The Friday ending a date's week
=A2+MOD(6-WEEKDAY(A2),7)
That week's last market date
=MAXIFS($A$2:$A$2941,$A$2:$A$2941,"<="&DATE(2025,4,18))
```

"MOD(n,7)" is the remainder after dividing by 7, here the days from the date to its Friday. "WEEKDAY" without its 2 counts Sunday as 1, as notebook 03 showed.

Two Tests for the Labels

```
def read_treasury() -> pd.DataFrame:
    return pd.read_csv(HERE / "treasury_par_yields.csv", parse_dates=["date"], index_col="date")
def test_month_end_label_is_observation_date():
    ten = read_treasury()["10 Yr"]
    month_end = md.last_per_period(ten, "ME")
    assert month_end.loc["2026-09"].index[0] == pd.Timestamp("2026-09-30")
    assert month_end.loc["2026-09-30"] == 5.29
    assert pd.Timestamp("2026-05-29") in month_end.index and pd.Timestamp("2026-05-31") not in month_end.index
def test_good_friday_week_takes_thursday():
    ten = read_treasury()["10 Yr"]
    weekly = md.last_per_period(ten, "W-FRI")
    assert pd.Timestamp("2025-04-17") in weekly.index
    assert pd.Timestamp("2025-04-18") not in weekly.index
    assert weekly.loc["2025-04-17"] == ten.loc["2025-04-17"]
```

They read the real file, so each asserts a fact about real dates.

```
...............   [100%]
15 passed in 0.50s
```

4.6 The as-of value: value_as_of

A report runs on Labor Day, and a client asks for the 10-year as of a Saturday. The as-of value is the last observation on or before the date, and it should say which row it took. Course 1's Treasury Yields case study used "XLOOKUP" with -1 once, on a month end.

```
%%writefile -a marketdata.py
def value_as_of(series: pd.Series, as_of) -> tuple[pd.Timestamp, float]:
    """The date and value of the last observation on or before `as_of`.
    Units: the value has the series' units. Convention: the as-of value; on a holiday or weekend it is
    the previous market day's value. Excel: =XLOOKUP(as_of, dates, values, , -1).
    Raises ValueError if as_of is before the first observation, the series has missing values, or its
    index is not unique ascending dates.
    """
    _check_series(series, "series")
    as_of = pd.Timestamp(as_of)
    if as_of < series.index[0]:
        raise ValueError(f"{as_of.date()} is before the first observation, {series.index[0].date()}")
    position = series.index.searchsorted(as_of, side="right") - 1
    return series.index[position], float(series.iloc[position])
```

Append It, Docstring First

Read the docstring first. "tuple[pd.Timestamp, float]" says it returns two things, a date and a value, to unpack into two names as Course 1's loops unpacked pairs. searchsorted(as_of, side='right') is where the date would go among the sorted dates, after an equal one, so one less is the last row on or before it.

Labor Day, a Saturday, a Market Day

```
importlib.reload(md)
for as_of in ['2026-09-07', '2026-10-03', '2026-09-30']:
    as_of_date, value = md.value_as_of(ten, as_of)
    print(f'as of {as_of}: {value} from {as_of_date.date()}')
as of 2026-09-07: 4.78 from 2026-09-04
as of 2026-10-03: 5.28 from 2026-10-02
as of 2026-09-30: 5.29 from 2026-09-30
```

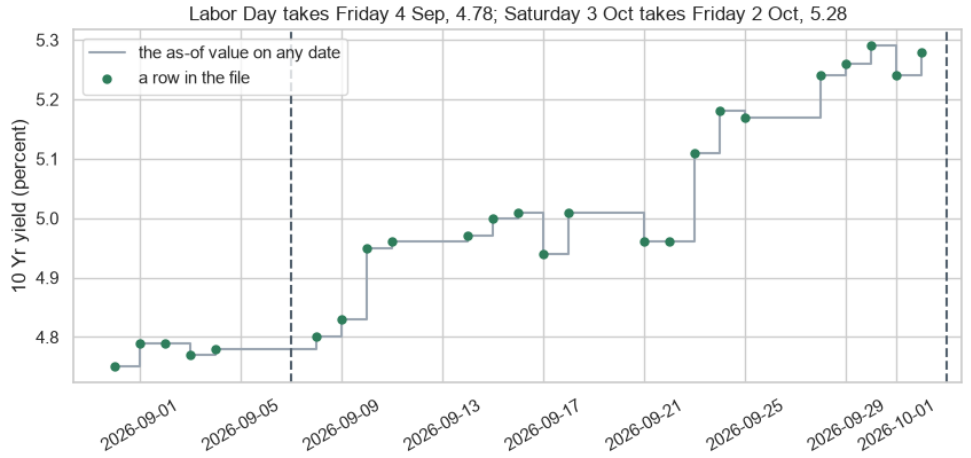
Excel side by side: XLOOKUP with -1, and the steps

Excel side by side.

Job	Python	Result
As of Labor Day	`md.value_as_of(ten, '2026-09-07')`	4.78
The date it came from	`its first item`	2026-09-04
As of Saturday 3 October	`md.value_as_of(ten, '2026-10-03')`	5.28

```
As of Labor Day
=XLOOKUP(DATE(2026,9,7),$A$2:$A$2941,$M$2:$M$2941,,-1)
The date it came from
=XLOOKUP(DATE(2026,9,7),$A$2:$A$2941,$A$2:$A$2941,,-1)
As of Saturday 3 October
=XLOOKUP(DATE(2026,10,3),$A$2:$A$2941,$M$2:$M$2941,,-1)
```

Match mode -1 is the exact date or the next smaller one: the as-of value.



The as-of value is the last row on or before the date, returned with that row's date: Labor Day's is Friday's 4.78.

4.7 Before the first row, and a column with gaps

The file starts on 2015-01-02. A report can still ask for a date before it, or read a column that is mostly empty.

value_as_of, Before the File Starts

```
md.value_as_of(ten, '2014-12-31')
ValueError: 2014-12-31 is before the first observation, 2015-01-02
```

Excel side by side. "=XLOOKUP(DATE(2014,12,31),$A$2:$A$2941,$M$2:$M$2941,,-1)" shows "#N/A": no row on or before it.

A Column With Gaps

```
md.value_as_of(treasury['1.5 Month'], '2026-09-07')
ValueError: series has 2532 missing values; drop them first with .dropna()
```

```
print(md.value_as_of(treasury['1.5 Month'].dropna(), '2026-09-07'))
(Timestamp('2026-09-04 00:00:00'), 3.83)
```

"value_as_of" never drops anything itself, so a gap is never hidden.

Two Tests for the As-Of Value

```
def test_value_as_of_holiday_returns_previous_day():
    ten = read_treasury()["10 Yr"]
    as_of_date, value = md.value_as_of(ten, "2026-09-07")
    assert as_of_date == pd.Timestamp("2026-09-04")
    assert value == ten.loc["2026-09-04"]
def test_value_as_of_before_start_raises():
    ten = read_treasury()["10 Yr"]
    with pytest.raises(ValueError):
        md.value_as_of(ten, "2014-12-31")
```

```
.................   [100%]
17 passed in 0.50s
```

4.8 Month-end levels for 2026

The 10 Yr and 2s10s at Each Month End

```
curve = pd.DataFrame({'10 Yr': ten, '2s10s_bp': (ten - treasury['2 Yr']) * 100})
month_ends = md.last_per_period(curve, 'ME').loc['2026']
month_ends.round(2)
```

Excel side by side. On the "MAXIFS" dates in column R, "=(XLOOKUP(R1,$A$2:$A$2941,$M$2:$M$2941)-XLOOKUP(R1,$A$2:$A$2941,$I$2:$I$2941))*100" filled down equals the 2s10s on all 142 months, with a largest gap of 0.0 bp.

A Pivot Table Grouped by Month

A pivot table is the other way Excel gives one number per month.

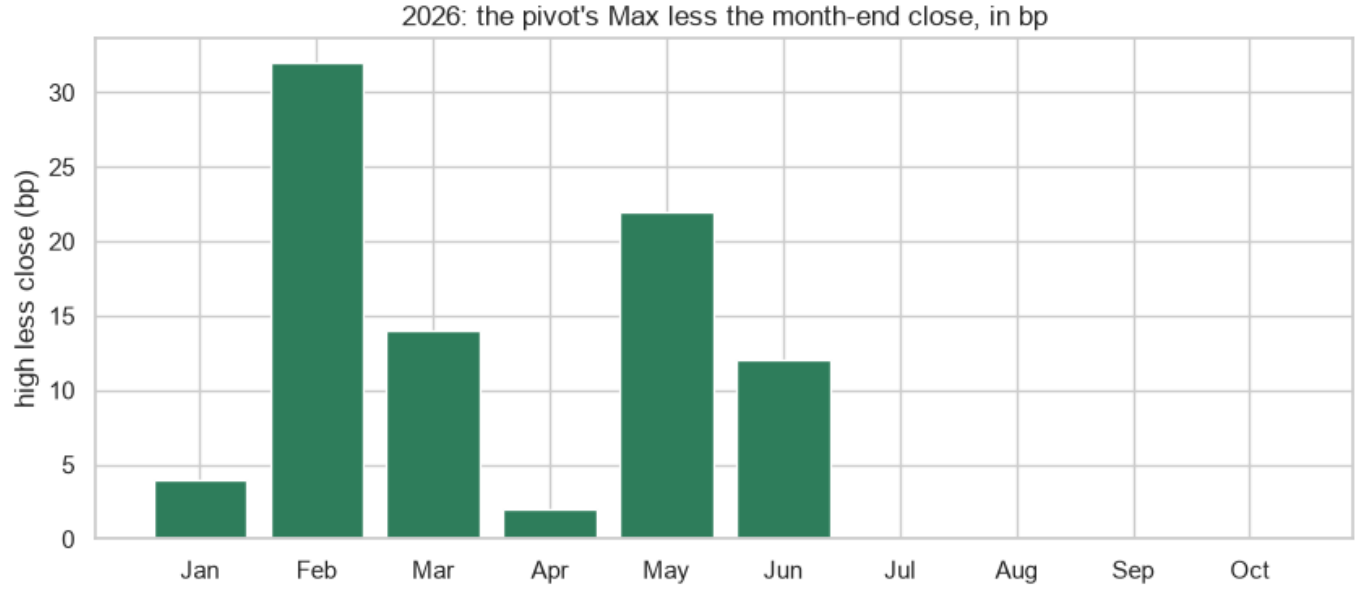
Job	Excel	Result
Each month's last market date	Insert, PivotTable on the 2026 rows, date in Rows grouped by Months,	the ten dates above
	date in Values set to Max	
The pivot's number for the 10 Yr	The 10 Yr in Values, set to Max	the month's high: 4.3 in January

```
Each month's last market date
month_ends.index
The pivot's number for the 10 Yr
daily_2026.groupby(daily_2026.index.month).max()
```

The pivot's Max against the month-end close

Each Month's High Against Its Close

```
highs = daily_2026.groupby(daily_2026.index.month).max()
gap_bp = (highs.to_numpy() - month_ends['10 Yr'].to_numpy()) * 100
print((gap_bp > 0).sum(), 'of', len(highs), 'months: the high is above the close')
6 of 10 months: the high is above the close
```



Take the dates from the pivot and the values with "XLOOKUP": a pivot's Max of a yield is the month's high.

Ten month ends, each on a market day, the 10 Yr and 2s10s equal to Excel; a pivot's Max is the month's high, not its close, in 6 of 10 months.

4.9 Commit the day, and an idea half done

The note's last ask: keep any half-finished idea out of the commit.

Commit the Day

```
!git -C "{work}" status --short
!git -C "{work}" diff --stat
 M marketdata.py
 M test_marketdata.py
 marketdata.py   | 51 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 37 ++++++++++++++++++++++++++++++++++++
 2 files changed, 88 insertions(+)
```

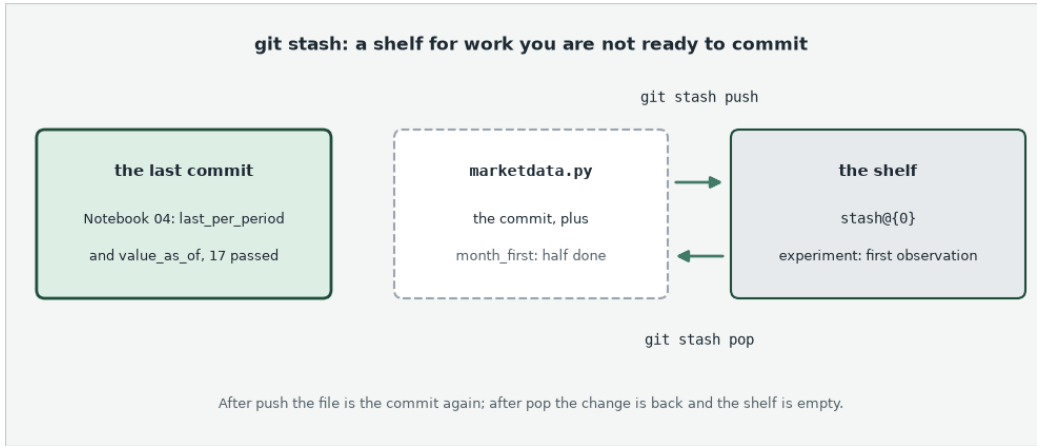
```
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "Notebook 04: last_per_period and value_as_of"
!git -C "{work}" log --oneline
9f01b1c Notebook 04: last_per_period and value_as_of
a439502 Notebook 03: missing_weekdays, align, and stale_runs
ceea769 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

A colleague wants the month-end table now, from the tested module. "git stash" puts the change on a shelf and hands back the file as committed.

An Idea, Half Done

Q. What if a month were stood for by its first observation? You start trying it in the module.

```
%%writefile -a marketdata.py
# experiment, not finished: a month stood for by its first observation?
def month_first(series: pd.Series) -> pd.Series:
    """The first observation of each calendar month (an experiment)."""
    return series.resample("ME").first()
```



Put it on the shelf, take it back, discard it

Put It on the Shelf

```
!git -C "{work}" stash push -m "experiment: first observation of each month"
!git -C "{work}" stash list
!git -C "{work}" status --short
Saved working directory and index state On main: experiment: first observation of each month
stash@{0}: On main: experiment: first observation of each month
```

The Tested Module, Untouched

```
.................   [100%]
17 passed in 0.50s
```

```
importlib.reload(md)
print('month_first in the module:', hasattr(md, 'month_first'))
md.last_per_period(ten, 'ME').loc['2026-07':'2026-09']
month_first in the module: False
date
2026-07-31   4.75
2026-08-31   4.75
2026-09-30   5.29
Name: 10 Yr, dtype: float64
```

"git stash" parks half-done work and hands back the committed file; "pop" returns it, and nothing half-done reaches a commit.

Take It Back

```
!git -C "{work}" stash pop -q
!git -C "{work}" stash list
!git -C "{work}" diff --stat
 marketdata.py | 6 ++++++
 1 file changed, 6 insertions(+)
```

The shelf is empty and the change is back. 4.1 already answered it: the first observation is the month's open, the wrong edge.

Discard It

```
!git -C "{work}" restore marketdata.py
!git -C "{work}" status --short
!git -C "{work}" log --oneline
9f01b1c Notebook 04: last_per_period and value_as_of
```

"git stash drop" throws a shelved change away without bringing it back.

Bring it home: into my_bondmath

Copy the work folder's two files into "my_bondmath", with the Treasury file the tests now read, test them, and commit. The copy replaces your two files, so anything of your own must be in the work folder first. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_04_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
Copy-Item "$work\treasury_par_yields.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py
git commit -m "Notebook 04: last_per_period and value_as_of"
```

pytest shows "17 passed", and "git status" lists the Treasury file as "??": it stays out of the commit. In Colab, download the two files from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Say which observation stands for a month or a week, and why the course takes the close, not the mean.

Read resample's period-end labels, and find the ones the file has no row for.

Take each period's last row with its own date, "last_per_period", tied to "MAXIFS" and "XLOOKUP".

Show a week with no Friday, labelled by its Thursday.

Give the value as of any date, with the date it came from, "value_as_of", tied to "XLOOKUP" with -1.

Park half-done work with "git stash", and bring it back with "git stash pop".

Predict first: the 2026 labels, and weeks without a Friday

Predict 1

That line gives ten rows for 2026. How many of those ten labels are dates the Treasury file has no row for? Write down a number.

```
by_resample = ten.resample('ME').last()
labels_2026 = by_resample.loc['2026'].index
for label in labels_2026:
    print(label.date(), label.strftime('%A'), label in ten.index)
off_file = labels_2026.difference(ten.index)
print(len(off_file), 'of the', len(labels_2026), 'labels have no row')
```

Predict 2

Good Friday 2025's week ends on a Thursday. How many of the file's 614 weeks end on a day other than Friday?

```
not_friday = weekly.index[weekly.index.dayofweek != 4]
print(len(not_friday), 'of', len(weekly), 'weeks end before Friday')
not_friday.strftime('%A').value_counts()
```

A) 8: the Good Fridays

B) 20

C) 126: every weekday with no row

Predict first: a grid of Saturdays, and asof before the file

pandas has a second way to change frequency: "asfreq".

Predict 3

ten.asfreq('W-SAT') puts the 10 Yr on a grid of Saturdays. How many Saturdays come back with a value?

A) nearly all: each takes Friday's

B) none

C) the call stops with an error

```
saturdays = ten.asfreq('W-SAT')
print(len(saturdays), 'Saturdays,', saturdays.count(), 'with a value')
```

Predict 4

pandas has the same idea built in, "ten.asof(date)", which returns the value only. Ask it for 2014-12-31, before the file starts. It returns:

A) a "ValueError"

B) "nan"

C) the first value, 2015-01-02's

```
print(ten.asof(pd.Timestamp('2026-09-07')), ten.asof(pd.Timestamp('2014-12-31')))
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: month_end_levels's test, written from its docstring, fails: AssertionError: September 2026 is 4.79, not 5.29.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
Which observation stands for a month or a week	The last: 5.29 on 2026-09-30 for September
The date beside it	last_per_period: 142 months, every label a market day (43 of resample's are not)
The value as of any date	value_as_of: Labor Day gives Friday's 4.78, and says so
The 2026 month ends	The 10 Yr and 2s10s, equal to Excel's MAXIFS and XLOOKUP
Two tested functions, nothing half-done committed	17 passed, the experiment stashed and discarded

Be proud: every weekly and monthly number you report now carries the date it really comes from.

Next: The Market Dataset case study (notebook 05) caches FRED's series and builds the course's daily dataset.